

Docker et Spring Boot

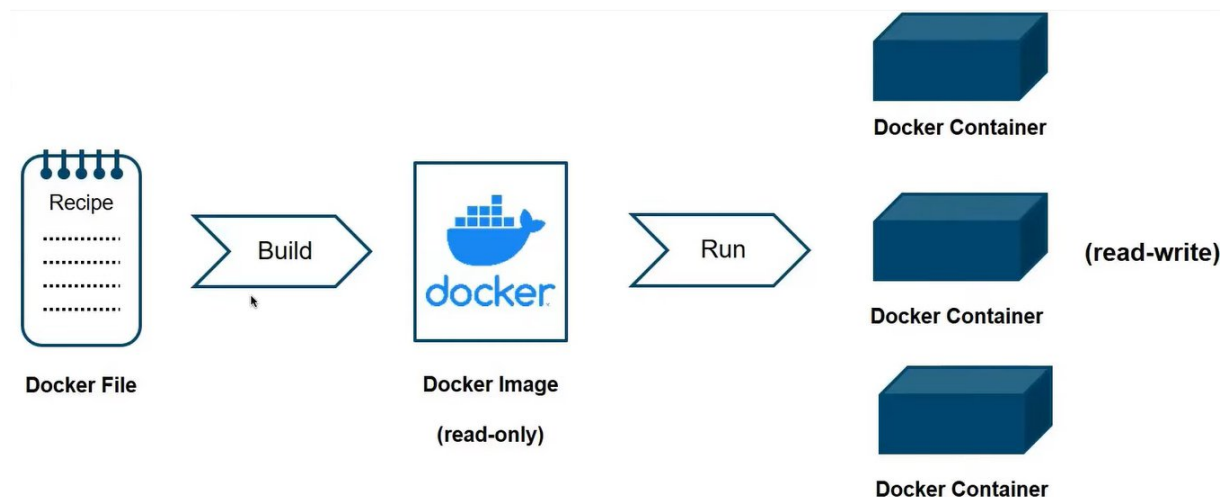
Explications Docker :

Au préalable, installer Docker Desktop : <https://www.docker.com/products/docker-desktop/>

Docker File : liste d'instructions qui permet de construire une image.

Docker Image : un modèle de notre application qui va emballer toutes les dépendances dont l'application a besoin pour fonctionner de manière autonome.

Docker Container : une instance isolée dans laquelle va s'exécuter l'application.



Création de l'application spring-boot-docker :

Dans IntelliJ, créer une application Spring Book avec Maven, Java 25, JAR et la dépendance Web pour ajouter un endpoint (API REST).

Ajouter un contrôleur, soit la classe GreetingController.java suivante :

```
package fr.olivier.springbootdocker;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting() {
        return "Greetings from Docker!";
    }

}
```

Lancer l'application :

```
package fr.olivier.springbootdocker;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class SpringBootDockerApplication {

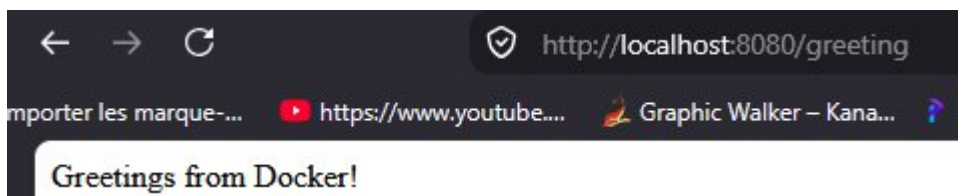
    public static void main(String[] args) {

        SpringApplication.run(SpringBootDockerApplication.class, args);

    }

}
```

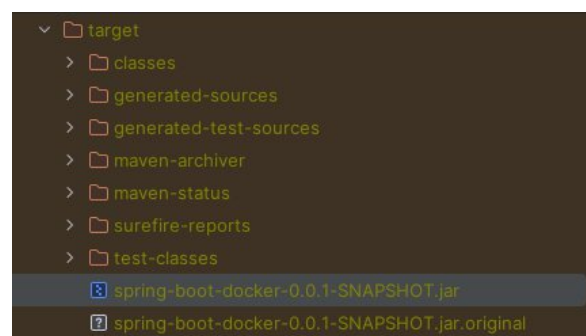
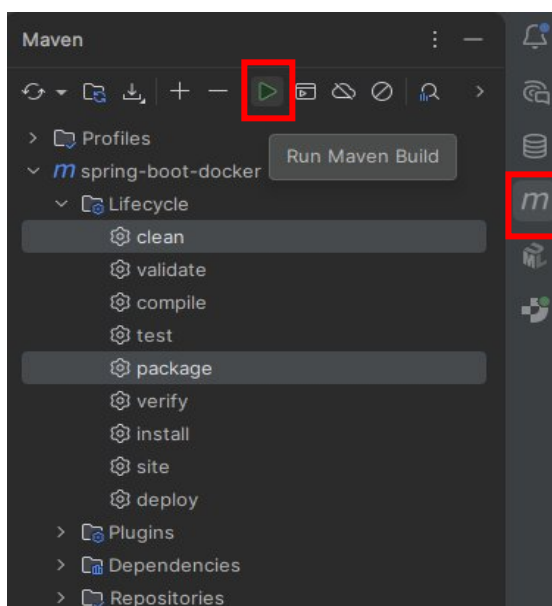
Vérifiez son fonctionnement avec le endpoint (<http://localhost:8080/greeting>) :



Maintenant, nous allons créer une image de cette application pour l'exécuter dans un container.

Création de l'image :

Il s'agit de pouvoir lancer un fichier jar de notre application. Pour le créer , lancer Maven : clean, package. Ce fichier apparaît dans target.



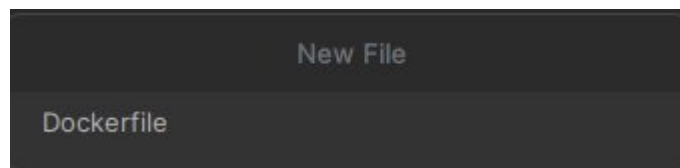
Ensuite dans le terminal :

```
java -jar target\spring-boot-docker-0.0.1-SNAPSHOT.jar
```



L'application est exécutée (vous pouvez à nouveau vérifier le endpoint dans votre navigateur). C'est ce que l'on va mettre en place avec Docker.

Dans le projet spring-boot-docker, crée un fichier Dockerfile (sans extension)



Avec votre navigateur, allez sur docker hub pour chercher une image d'eclipse-temurin, puis choisissez TAG **25-jdk** (pour windows)

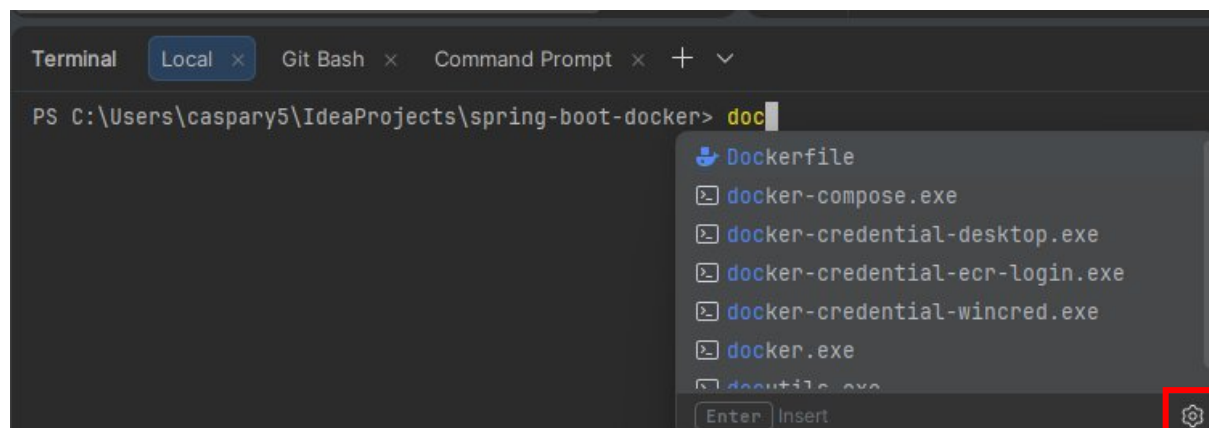
Soit : https://hub.docker.com/_/eclipse-temurin/tags?name=25-jdk

Compléter votre Dockerfile :

```
FROM eclipse-temurin:25-jdk
WORKDIR /app
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} /app/greeting.jar
ENTRYPOINT ["java", "-jar", "/app/greeting.jar"]
```

Le fichier .jar (il n'y en a qu'un dans target) sera renommé en greeting.jar dans le répertoire app de votre image.

Vérifiez que Docker est bien installé en tapant docker dans votre terminal, ainsi que la **complétion** (sinon voir settings) :



Créez votre image depuis votre projet :

```
docker build -t spring-boot-docker .
```

Pour vérifier sa création :

```
docker images
```

```
Terminal Local x + v
PS C:\Users\caspar5\IdeaProjects\spring-boot-docker> docker images

IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA
spring-boot-docker:latest  90d8a7560c3d      623MB         168MB
```

Maintenant, il faut créer un conteneur pour exécuter cette image :

D'abord, vérifions s'il existe des conteneurs avec la commande :

```
docker ps
```

```
PS C:\Users\caspar5\IdeaProjects\spring-boot-docker> docker ps

CONTAINER ID    IMAGE                COMMAND         CREATED        STATUS        PORTS        NAMES
```

Ici, il n'y en a pas. Pour créer le conteneur, on ajoute les ports nécessaires et le nom de l'image :

```
docker run -p 80:8080 spring-boot-docker
```

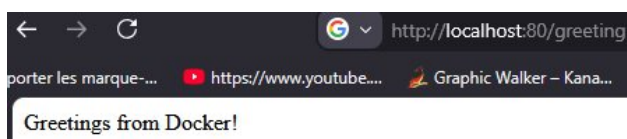
le port 80 sera maintenant le port actif à la place du port 8080.

Ouvrez un autre terminal et vérifiez que le conteneur existe avec la commande `docker ps` :

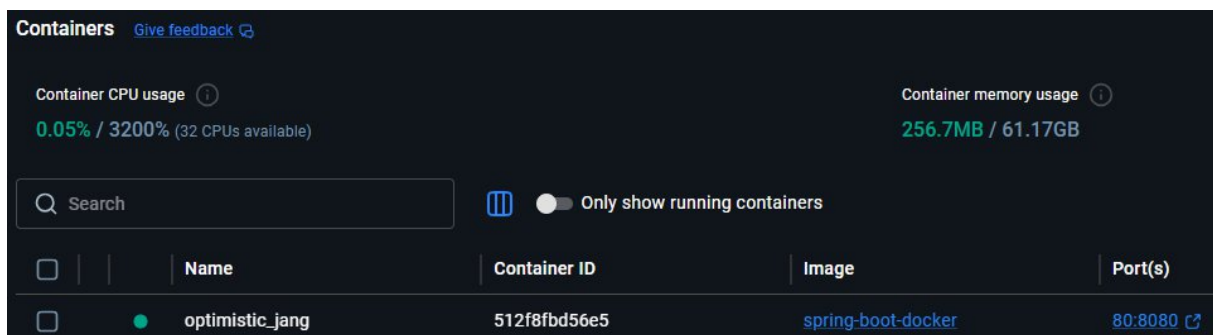
```
PS C:\Users\caspar5\IdeaProjects\spring-boot-docker> docker ps

CONTAINER ID    IMAGE                COMMAND         CREATED        STATUS        PORTS        NAMES
512f8fd56e5    spring-boot-docker   "java -jar /app/gree..." 3 minutes ago  Up 3 minutes  0.0.0.0:80->8080/tcp, [::]:80->8080/tcp
```

Vérifier le fonctionnement dans le navigateur :



Le conteneur est également visible dans Docker Desktop :



Quand on arrête l'application (Ctrl+C), il n'y a plus de conteneur, vérifier avec **docker ps**, mais l'image reste : **docker images**.

Supprimer l'image qui a été créée (-f force la suppression) :

```
docker rmi spring-boot-docker -f
```

vérifier avec docker images.

Recommencez le processus (création d'une image, lancer un conteneur) avec le Dockfile suivant qui intègre maven :

```
FROM maven:3.9.10-eclipse-temurin
WORKDIR /app
COPY . .
RUN mvn clean package
FROM eclipse-temurin:25-jdk
WORKDIR /app
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} /app/greeting.jar
ENTRYPOINT ["java", "-jar", "/app/greeting.jar"]
```

Vous pouvez aller sur Docker Hub pour trouver Maven et le Tag.

La commande `mvn --version` permet de voir la version installée sur votre ordinateur si c'est le cas.

Recommencer successivement :

```
docker build -t spring-boot-docker .
docker images
docker run -p 80:8080 spring-boot-docker
docker ps
```

Publication de l'application :

Se connecter à Docker (installé préalablement sur votre machine), dans le terminal :

```
docker login
```

```
PS C:\Users\caspar5\IdeaProjects\spring-boot-docker> docker login
Authenticating with existing credentials... [Username: ocasper]

i Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
```

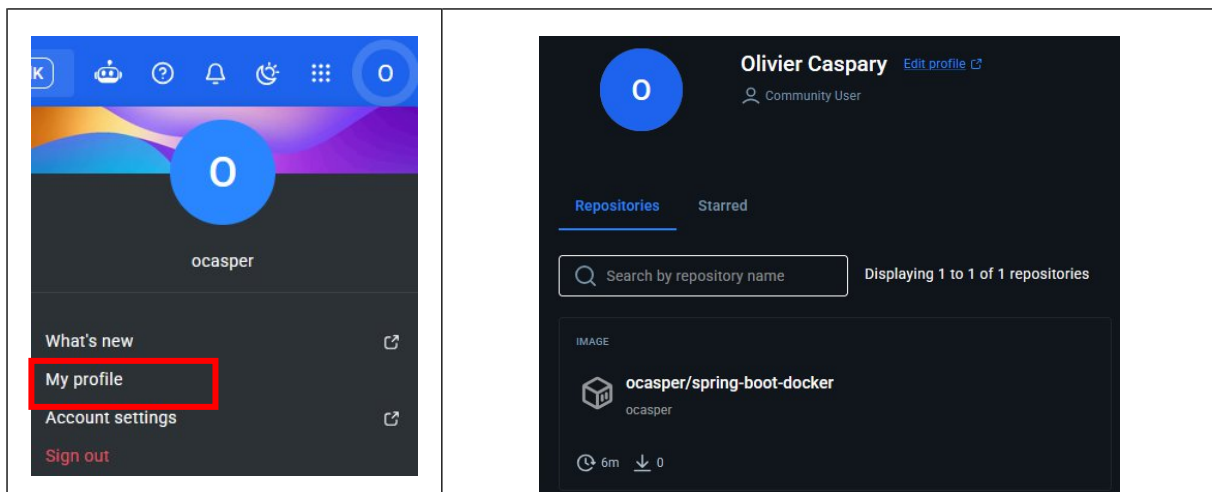
Après `docker tag`, mettre le nom de votre image, votre nom d'utilisateur, suivi de `/votre version:tag de votre version` :

```
docker tag spring-boot-docker ocasper/spring-boot-docker:1.0
```

Ensuite, poussez votre image vers Docker Desktop :

```
docker push ocasper/spring-boot-docker:1.0
```

Vérifier sur Docker Hub, votre profil :



Tout le monde peut y accéder et exécuter le code sur sa machine car l'image est empaquetée avec toutes ses dépendances.

Dans le terminal, pour lancer l'image « conteneurisée » :

```
docker run -p 80:8080 ocasper/spring-boot-docker:1.0
```

Ouvrez un autre terminal local, et faites :

```
docker images
```

```
PS C:\Users\caspar5\IdeaProjects\spring-boot-docker> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ocasper/spring-boot-docker:1.0	082cadefa9ca	623MB	168MB	U
spring-boot-docker:latest	082cadefa9ca	623MB	168MB	U

Vous obtenez votre image locale et celle provenant de votre compte Docker.

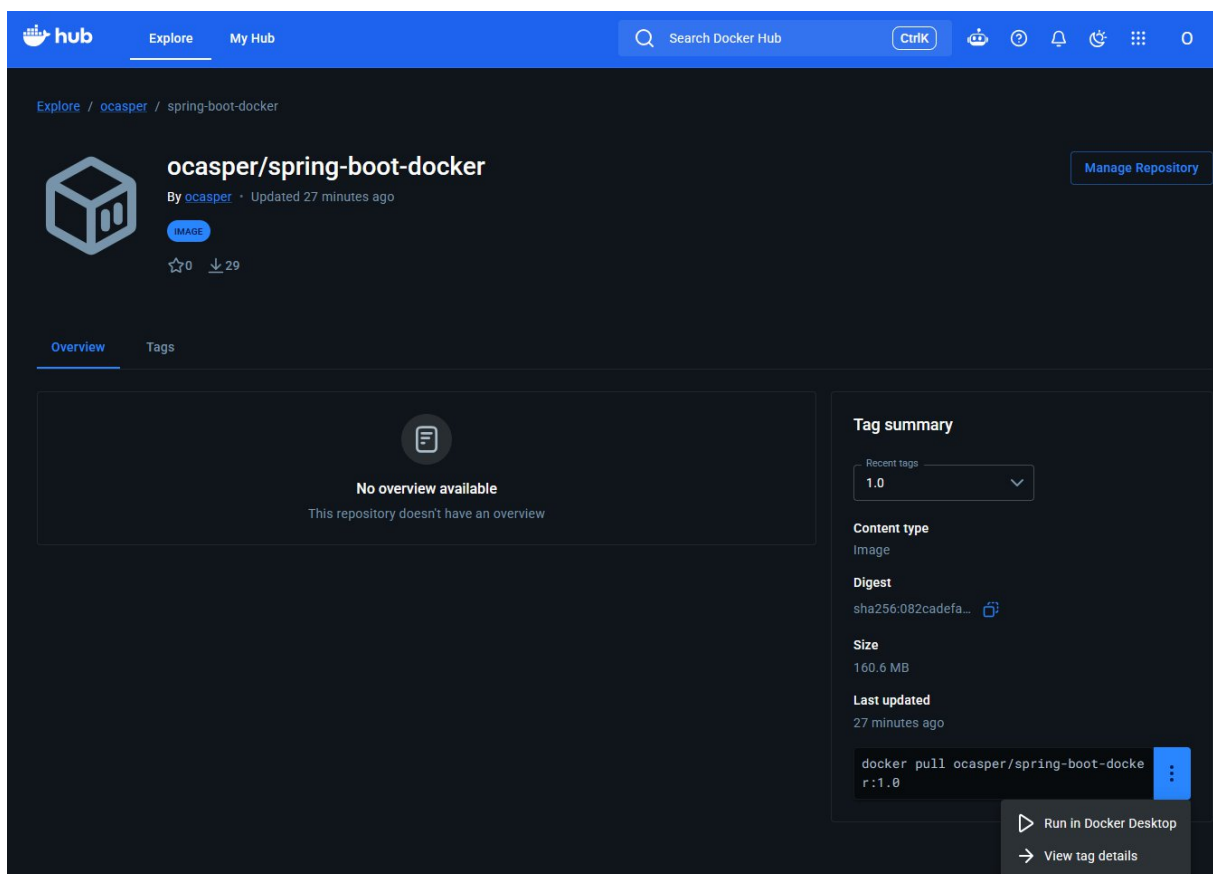
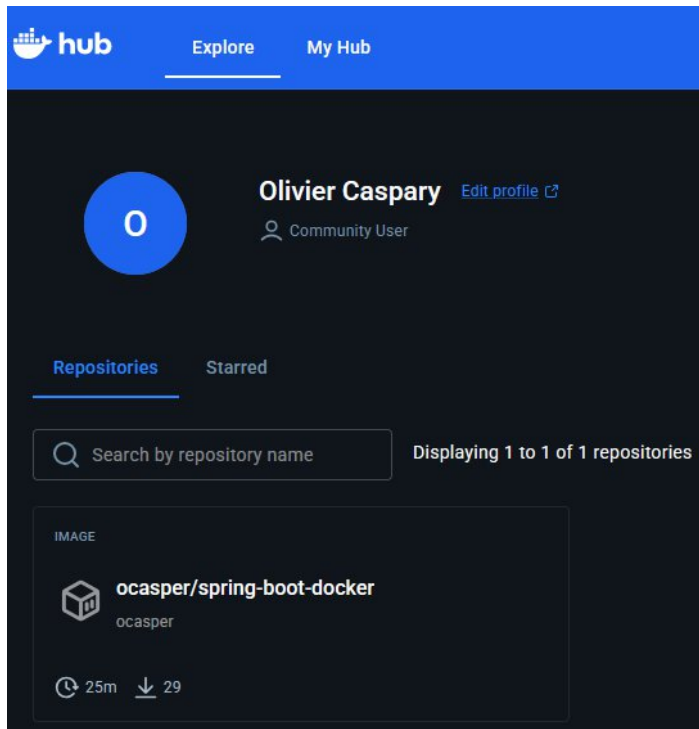
A noter que votre application peut se lancer directement depuis votre compte Docker Desktop :



Pour récupérer l'application :

```
docker pull ocasper/spring-boot-docker:1.0
```

Sur Docker Hub :



Référence : <https://www.youtube.com/watch?v=IzM9-g6en3Q&t=922s>